The Chinese University of Hong Kong

CENG3430 Rapid Prototyping of Digital Systems

Final Project—FPGA Implementation of CRYSTALS-Kyber Key Generation

Wong Cheuk Yin Student ID: 1155192671

May 15, 2026

Source code available on GitHub:

https://github.com/TobiaKW/FPGA-Implemented-CRYSTALS-Kyber768-Key-Generation

Contents

1 Introduction **2**
1.1 System goal and motivation . . . 2
1.2 Scope . . . 2

2 Design **2**
2.1 Overview . . . 2
2.2 External modules adopted . . . 2
2.2.1 neoTRNG . . . 2
2.2.2 Hash Server . . . 2
2.2.3 kyber-polmul-hw . . . 3
2.2.4 Simulation waveforms . . . 3
2.3 Overall dataflow . . . 4

3 Implementation **5**
3.1 Module descriptions . . . 5
3.1.1 `a_gen` FSM . . . 5
3.1.2 `se_gen` FSM . . . 5
3.1.3 `hash_unit` FSM . . . 5
3.1.4 `mat_vec_mul` FSM . . . 6
3.1.5 `bram_sdp_12x768` . . . 6
3.1.6 `topserver` (orchestrator) . . . 6
3.1.7 `topserver_axi` . . . 6
3.2 Block Design . . . 6
3.3 PYNQ script . . . 7

4 Results and Discussion **7**
4.1 Preset goals . . . 7
4.2 Difficulties and bottlenecks . . . 7
4.2.1 LUT over-utilisation. . . . 7
4.2.2 MMIO / optimisation datapath. . . . 8
4.3 Further improvements . . . 8

5 Declarations **9**
5.1 AI tools . . . 9
5.2 References . . . 9

1 Introduction

1.1 System goal and motivation

Since it is a open topic project, I would like to design a system that can utilized the Logic resources of FPGA and the strength of PL compared to software. Post-quantum cryptography (PQC) is a natural fit because higher algorithmic complexity can amplify the benefit of hardware acceleration. Among PQC schemes, **CRYSTALS-Kyber** (KEM) was chosen because it is widely deployed, the RTL structure is relatively approachable, and reference material and open-source Kyber-related hardware exist online.

1.2 Scope

The initial plan was three servers—keygen, encapsulation, and decapsulation—using open-source blocks. Device LUT capacity on the ZedBoard proved insufficient for that breadth, so the project was narrowed to a **single server: key generation only**.

2 Design

2.1 Overview

The mathematical target of this keygen algorithm is:

$$\mathbf{t} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e} \pmod{3329}$$

$$\text{pk} = (\rho \mid\mid \mathbf{t})$$

$$\text{sk} = (\mathbf{s} \mid\mid \text{pk} \mid\mid H(\text{pk}) \mid\mid z).$$

TRNG engine generates ρ, σ, z as the seeds for the keygen algorithm.
A is a 3x3 matrix of polynomials , with every element is a polynomial of degree 255 with coefficients in range [0, 3328].
s, **e** are 3x1 noise vectors, with every element is a polynomial of degree 255 with coefficients in range [-2, 2].
pk and sk are the public and secret keys respectively, which are the output of the keygen algorithm.

2.2 External modules adopted

2.2.1 neoTRNG

[1] Outputs one random byte per valid cycle from on-chip entropy; drive it with clock and reset, assert `enable_i`, and latch `data_o` when `valid_o` is high. It is used in `topserver` during the TRNG states to collect ρ, σ, and z.

2.2.2 Hash Server

[2] `hash_core_Server.v` implements the Keccak sponge with FIFOs for SHAKE128 squeeze, SHA3-256 digest, and related modes. `hash_unit` asserts `start`, selects `mode`, and runs absorb/stream handshakes on the wrapper FSM. The same core is shared from `hash_unit` by `a_gen` (matrix expansion) and `topserver` (hash of the packed public key).

2.2.3 kyber-polmul-hw

[3] Performs 255-degree polynomial multiplication in the ring R_q (mod 3329) inside NTT domain. Any degree higher than 255 is folded into the lower degrees with the rule of $x^{256} = -1$. Used in `mat_vec_mul`: load matrix and vector coefficients, pulse the PE stages, and wait on `pe_done` as its own FSM defines. Performs $\mathbf{A} \cdot \mathbf{s}$ in the keygen datapath.

2.2.4 Simulation waveforms

The following screenshots illustrate post-RTL simulation for parts of the keygen datapath.

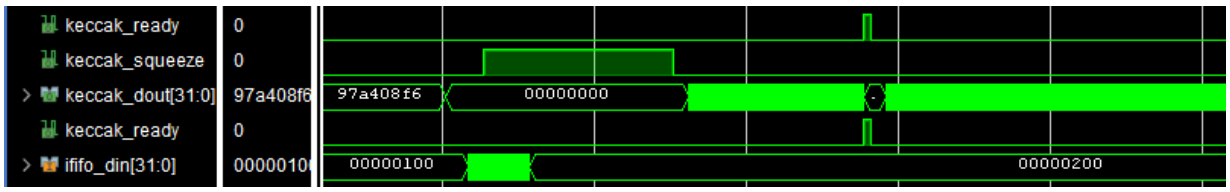

Figure 1: Keccak/`hash_core_Server`: `keccak_squeeze` enables the squeeze phase; `keccak_dout` streams 32-bit words while `keccak_ready` pulses; `ififo_din` shows absorb-side activity.

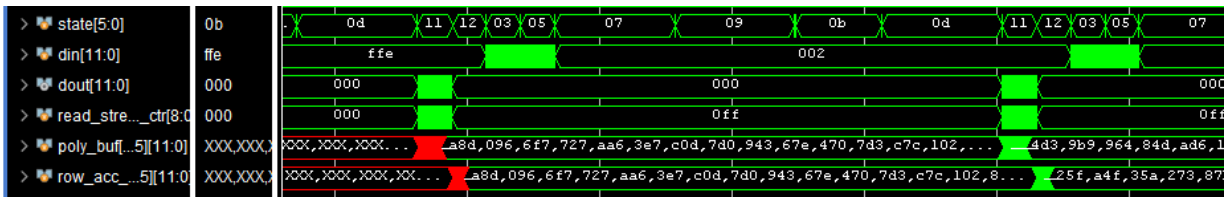

Figure 2: Polynomial multiply (`mat_vec_mul` / PE): `state` FSM progression, `din`/`dout`, read counter, and filling of `poly_buf` and `row_acc_buf` during a multiply–accumulate segment.

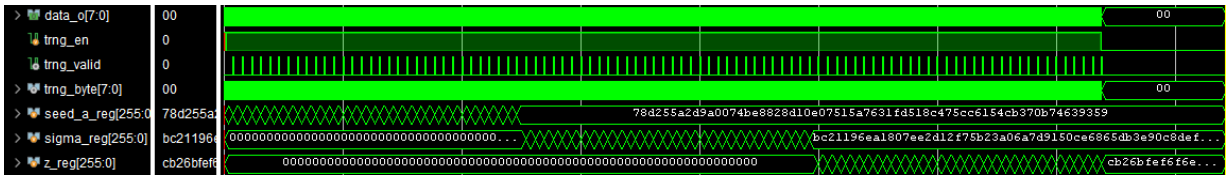

Figure 3: TRNG seed collection in `topserver`: `trng_en` gates sampling; `trng_valid` strobes per byte; `seed_a_reg`, `sigma_reg`, and `z_reg` latch the 256-bit seeds over time.

2.3 Overall dataflow

1. Init
2. Phase 1: generate seeds
   - 2.2.1 Initialise neoTRNG
   - 2.2.2 Collect random seeds into ρ, σ, z
3. Generate polynomials / matrix
   - 3.3.1 `a_gen`: For each element $A[i][j]$, SHAKE128($\rho \parallel j \parallel i$) to generate the coefficient.
   - 3.3.2 `se_gen`: noise vectors (LFSR demo; not the official method—official CBD/PRF was too LUT-heavy)
4. Computation of t
   - 4.4.1 Compute $\mathbf{As}$ (mod 3329)
   - 4.4.2 Add $\mathbf{e}$ (mod 3329)
5. Packaging
   - 5.5.1 Form `pk`
   - 5.5.2 SHA3-256(`pk`)
   - 5.5.3 Form `sk`

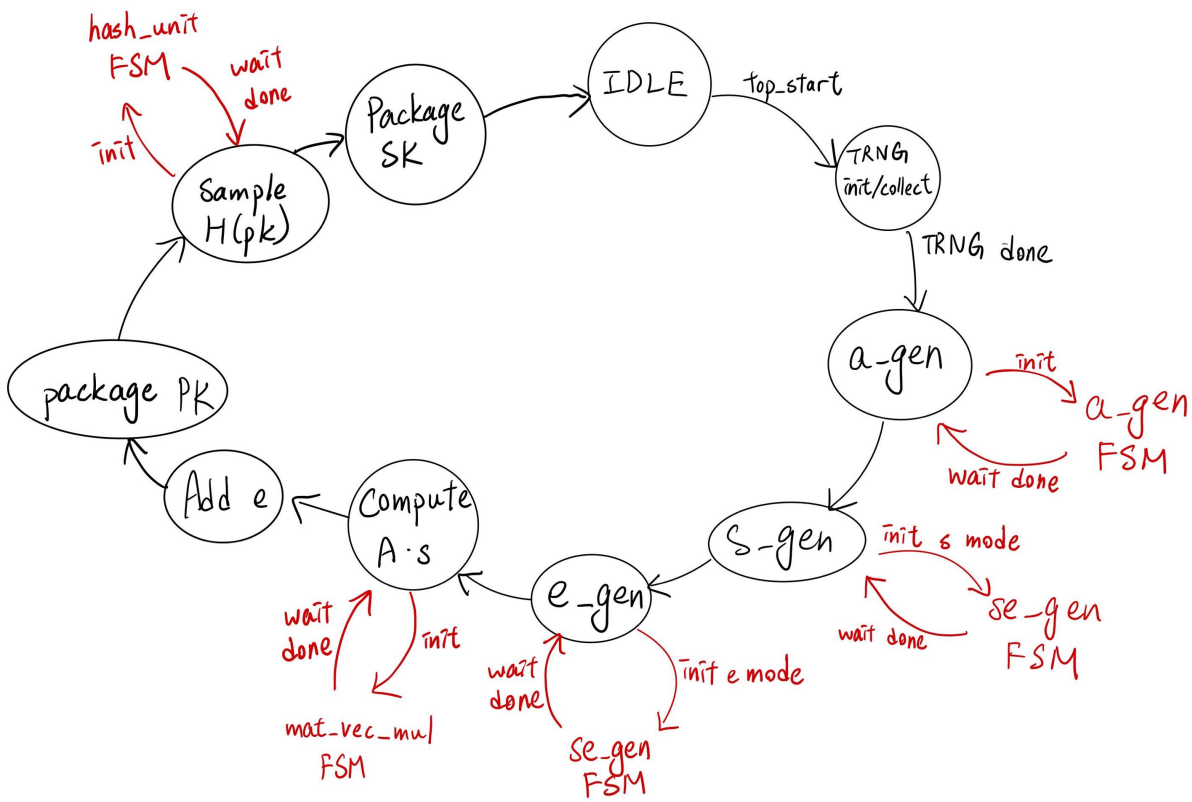

Figure 4: brief overview of the FSMs of the system. (hand-drawn)

3 Implementation

3.1 Module descriptions

3.1.1 `a_gen` FSM

Fills the flattened matrix memory A: nine polynomials each n=256 coefficients in range [0, 3328], one matrix cell at a time.
For each cell it uses the shared `hash_unit` in uniform-A mode with seed ρ and indices row/column of the current cell.

- **`ST_IDLE`:** detects `a_gen_start` pulse, reset row/column/cell counters, pulse `hash_start_o`, go to `ST_SAMPLE`.
- **`ST_SAMPLE`:** While `hash_stream_valid_i`, take two 12-bit candidates per 32-bit word; *reject* values ≥ 3329; accepted samples fill a per-cell buffer `a_poly[0..255]` until 256 coefficients are collected, then assert `hash_stop_stream_o` and move to `ST_WAIT_HASH_DONE`.
- **`ST_WAIT_HASH_DONE`:** Wait for `hash_done_i`; clear stop, then go to `ST_STORE_POLY`.
- **`ST_STORE_POLY`:** Copy `a_poly` into `A_mem` at the address base for the current cell, one coefficient per cycle. If all nine cells are done go to `ST_DONE`; else advance `cell_ctr` (and row/column), restart `hash_start_o`, return to `ST_SAMPLE`.
- **`ST_DONE`:** Pulse `a_gen_done`, deassert `busy`, return to `ST_IDLE`.

Note: A is a large memory, we cannot directly output the whole matrix at once because it consumes many output ports (bad for routing). So we use a read address to read single 12-bit coefficient at a time. This is why we have the `a_mem_rd_addr` and `a_mem_rd_data` ports.

3.1.2 `se_gen` FSM

States: `ST_IDLE` → `ST_FILL` → `ST_DONE` → `ST_IDLE`

In the official Kyber flow, the small noise vectors $\mathbf{s}$ and $\mathbf{e}$ are drawn using a *pseudorandom function* (PRF) built from an XOF on the secret seed σ (with a nonce per polynomial) and then *centered binomial sampling* (CBD) to map pseudorandom bits to coefficients in $\{-2, \ldots, 2\}$. Implementing that PRF+CBD path in RTL consumed too many LUTs for our target device, so `se_gen` uses an **LFSR-based filler** (using σ as seed) instead for demonstration; this is *not* algorithm-compliant noise.

Same as `a_gen`, we uses read address to read single 12-bit coefficient at a time. But we have a extra input to choose s-gen mode or e-gen mode to generate/read the noise vector.

3.1.3 `hash_unit` FSM

States: `ST_IDLE` → `ST_INIT_WAIT` → `ST_ABSORB` → `ST_WAIT_READY` → `ST_STREAM` → `ST_DONE` → `ST_IDLE`

Wraps the `hash_core_Server` to enable sharing of the same core different modules. Different modes are used to distinguish the different uses of the hash unit.

3.1.4 `mat_vec_mul` FSM

States: `ST_IDLE` → `ST_SETADDR` → `ST_LOAD_A_PULSE` → `ST_LOAD_A_STREAM` → `ST_LOAD_B_PULSE` → `ST_LOAD_B_STREAM` → `ST_FNTT_A_*` → `ST_FNTT_B_*` → `ST_PWM2_*` → `ST_INTT_*` → `ST_READ_PRE` → `ST_READ_A_*` → `ST_READ_STREAM` → `ST_ACC_ROW` → `ST_WRITE` → `ST_NEXT` → `ST_DONE` → `ST_IDLE`

Computes $\mathbf{y} = \mathbf{A} \cdot \mathbf{s}$ with the use of kyber-polmul-hw/pe1 module. The FSM follows the standard procedure to drive the core and do accumulation for each cell in the product vector.

Pulse `start` to trigger one triple loop (for one cell in the product vector, 3 cells in total in the product vector). Matrix/vector coefficients are streamed one at a time; results go out on `out_wr_data` with the use of address.

3.1.5 `bram_sdp_12x768`

Simple dual-port synchronous template (one write port, one read port; read data registered, so **one-cycle read latency**). Instantiated as `u_t_mem` in `topserver` to hold the 768 coefficients of t (12-bit words). The `(* ram_style = "block" *)` attribute and regular read/write pattern steer Vivado to map `mem` to **Block RAM**, avoiding a large register/LUT implementation.

3.1.6 `topserver` (orchestrator)

Single large FSM; shares one `hash_unit` with `a_gen` via mux. Representative state sequence: `ST_IDLE` → `ST_TRNG_INIT/COLLECT` → `ST_WAIT_A_GEN` → `ST_WAIT_S_GEN` → `ST_WAIT_E_GEN` → `ST_COMPUTE_AS/WAIT_AS_DONE` → `ST_ADD_E` → build PK → hash PK → build SK (pack s, copy $H(\text{pk})$, z) → `ST_DONE` → `ST_IDLE`.

3.1.7 `topserver_axi`

AXI4-Lite slave: maps PS accesses to `CONTROL`, `STATUS`, `READ_CFG`, `READ_ADDR`, `READ_DATA`; instantiates `topserver` with reset derived from `s_axi_aresetn`.

3.2 Block Design

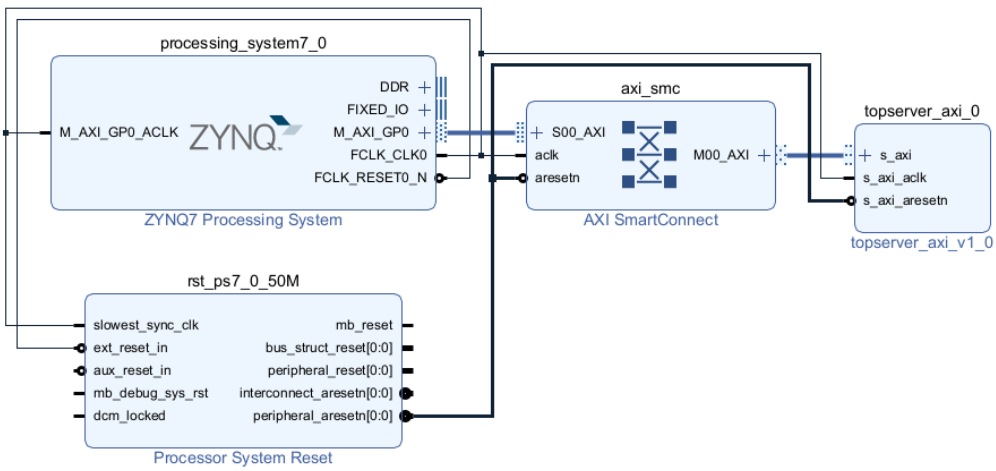

Figure 5: Vivado IP integrator block design: Zynq7 PS `M_AXI_GP0` through `axi_smc` to custom `topserver_axi_0`; `FCLK_CLK0` and `Processor System Reset` (`rst_ps7_0_50M`) supply clock and synchronized resets to the interconnect and PL peripheral.

3.3 PYNQ script

The board-side test flow lives in `pynq_script.py`. The script loads the Vivado `.bit` with PYNQ `Overlay`, opens an `MMIO` window at the `topserver_axi` base address, and drives the AXI-Lite registers defined in `topserver_axi.v`: it clears the sticky done flag, issues a one-shot `CONTROL` start pulse, then polls `STATUS` until keygen completes. After that it performs byte-by-byte reads by writing `READ_ADDR`, asserting `READ_CFG` (`rd_en` plus `rd_sk` to choose SK vs. PK), waiting for `rd_valid` in `STATUS`, and latching the low byte from `READ_DATA`. The full 2400-byte SK and 1184-byte PK buffers are assembled in Python and printed as hex dumps for quick verification on PYNQ.

4 Results and Discussion

4.1 Preset goals

The original full three-server goal was not met; keygen only was delivered in hardware, with demo noise and resource-driven trade-offs.

4.2 Difficulties and bottlenecks

4.2.1 LUT over-utilisation.

The ZedBoard offers about 53k LUTs; operating too close to the limit worsens routing and runtime (e.g. ∼50k LUTs leading to multi-hour implementation and poor slack). The final implementation is at about 48k LUTs, which is close to the limit.

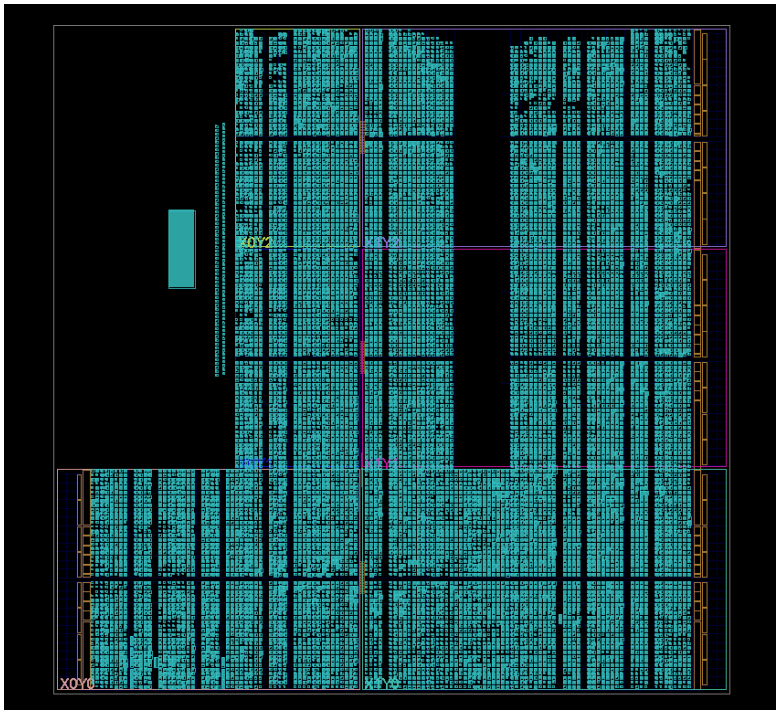

Figure 6: Implementated design of the system. Very full.

1. `se_gen` / noise generation.
Official Kyber-style sampling (CBD/PRF path) consumed very large LUT in this flow (about 38k LUT for this single module in my case). An LFSR-based filler reduced `se_gen` to about **86 LUTs** in the final routed build (with two RAMB18s for storage), at the cost of non-standard keys.

2. Polynomial multiplication.
A 16-PE pipeline from open-source Kyber polynomial-multiplier hardware pushed utilisation to roughly 100k LUTs (infeasible). The design was reduced to a **single-PE** pipeline to fit the device.

3. Shared hash core.
Multiple Kyber steps need hashing; duplicating the hash server per consumer would multiply LUT. A **shared** `hash_unit` with mode muxing serialises hash operations (no parallel hash).

4. Large arrays.
$\mathbf{A}$, $\mathbf{s}$, $\mathbf{e}$, $H(\mathrm{pk})$, `sk`, etc. are stored in **BRAM** inside modules with explicit read ports (12-bit read granularity $\Rightarrow$ software or FSM loops for full byte streams) in order to reduce the LUT usage.

5. Routing congestion.
With LUT resources highly utilized, the latency (WNS/TNS) of the system is not acceptable because of the routing congestion. To solve this issue, the frequency of the system is reduced from 100MHz to 50MHz to reduce the routing difficulty. In my case, the system with 50k/53k LUT usage 100MHz frequency runs implementation for 3 hours and get -2000ns TNS. but after switching to 50MHz, the implementation time is shortened to 1 hour and get positive TNS.

6. Fails to completely follow the official algorithm.
The official CRYSTALS-Kyber repo provide a KAT test vector to test the keygen algorithm. Since I did not follow the official algorithm strictly due to the LUT issue, the keygen algorithm does not pass the KAT test.

4.2.2 MMIO / optimisation datapath.

After optimisation, the AXI read path into `se_gen` appeared to collapse in the netlist, which is caused by the collapse of the AXI read path and overlay fails to read the data properly. Applying

```
(* keep_hierarchy = "yes", dont_touch = "yes" *)
```

on `se_gen` and its BRAM wrappers preserved the hierarchy and restored correct reads.

4.3 Further improvements

Finer modularisation (as in stronger reference projects), deeper algorithm/architecture planning, and a longer project cycle would improve completeness. Splitting into more packaged IP with clearer PS/PL boundaries would ease debug and maintenance; pushing more control to the PS can reduce PL congestion.

5 Declarations

5.1 AI tools

Cursor Copilot was used to help identifying the cause of some stubborn issues and debugging (partially), as some issues are beyond my knowledge. All testing and verification were done manually.

5.2 References

[1] S. Nolting, *neoTRNG* (VHDL): platform-independent true random number generator for FPGA/ASIC, with ring-oscillator entropy and post-processing. Repository: https://github.com/stnolting/neoTRNG.

[2] `@xingyf14`, *CRYSTALS-Kyber-FPGA-Implementation*: Verilog FPGA implementation of Kyber including the Keccak hashing core (`hash_core_Server`) adapted in this project. Repository: https://github.com/xingyf14/CRYSTALS-Kyber-FPGA-Implementation.

[3] B. Acmert, *kyber-polmul-hw*: hardware polynomial multiplication and NTT pipeline for Kyber (used via `KyberHPM1PE_top` in `mat_vec_mul`). Repository: https://github.com/acmert/kyber-polmul-hw.

[4] CRYSTALS consortium, *CRYSTALS-Kyber* / ML-KEM reference implementation and specification sources (algorithm definition, test vectors). Repository: https://github.com/pq-crystals/kyber.